Electrónica con ESP32: del circuito al primer programa

Reconocer la placa clásica y construir una salida LED cuya corriente pueda explicar.

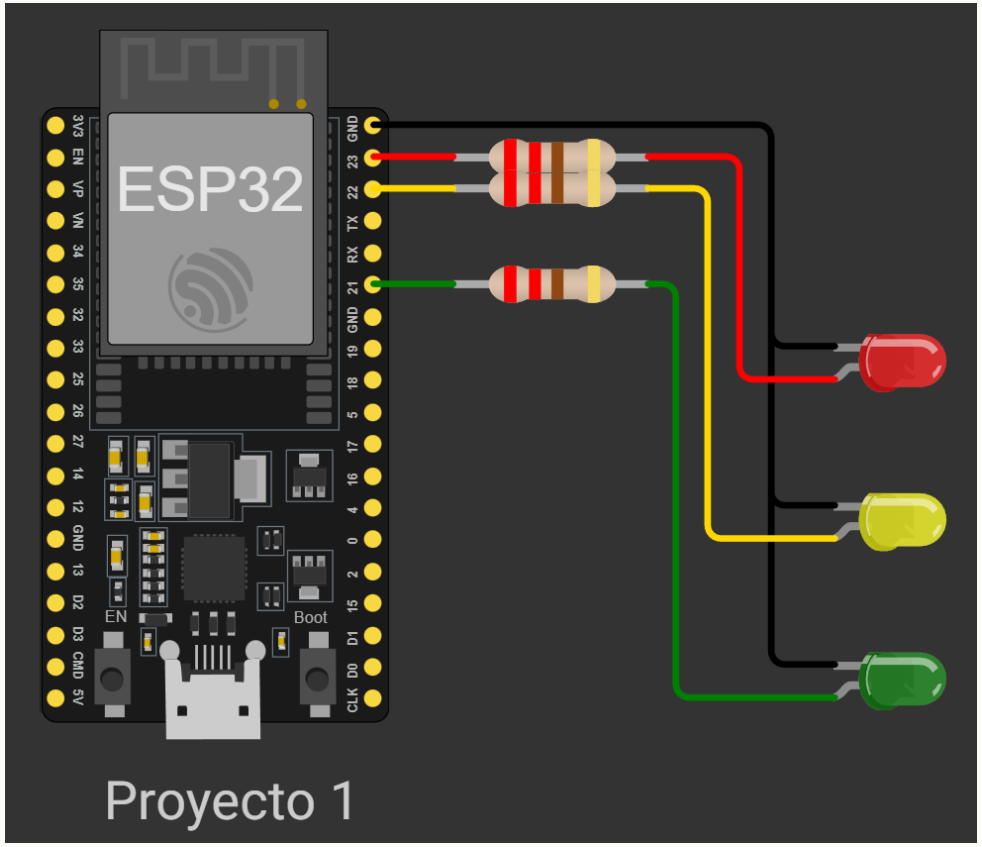
Un indicador de estado en una estación de drones debe comunicar una condición mediante un circuito y un programa que se entiendan.

EN LÍNEA 2 HORAS ESP32 CLÁSICO

Qué estamos construyendo

El primer objeto del curso es un LED externo con su resistencia. Parece sencillo, pero reúne las cuatro decisiones que repetiremos: de dónde sale la energía, por dónde circula la corriente, qué señal controla el comportamiento y cómo comprobaremos el resultado. El ESP32 es un microcontrolador: ejecuta instrucciones y dispone de periféricos para relacionarlas con entradas y salidas. La placa de desarrollo añade alimentación, conexiones y una interfaz de programación. El módulo metálico y el conector USB cumplen funciones diferentes.

Qué estamos construyendo



Sigue el camino GPIO → resistencia → LED → GND. ¿Dónde medirías la caída en la resistencia?

MECRO, proyecto 01. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Interpretación · Qué estamos construyendo

Identifica el texto del módulo antes de elegir la placa en el entorno. El curso usa ESP32 clásico, confirmado para el kit MECRO. Una placa S3 requiere otra selección y otro mapa de pines. En el protoboard, los grupos de contactos y los rieles de alimentación no son todos continuos. Con el circuito desconectado, sigue con el dedo el camino GPIO, resistencia, LED y GND. Si el camino termina en una fila sin continuidad, ningún cambio de código lo completará.

La referencia GND permite interpretar el voltaje de una señal. Medir 3,3 V significa medir una diferencia respecto a esa referencia; no significa que el pin pueda entregar cualquier corriente. Un GPIO representa una orden de baja potencia. Durante las primeras prácticas separaremos esa función de la energía que necesitan servos y motores. Esta distinción será fundamental al reconocer el controlador, los ESC y la batería de un dron.

Voltaje, corriente y resistencia con unidades

El voltaje expresa diferencia de potencial y se mide en voltios. La corriente expresa flujo de carga y se mide en amperios. La resistencia relaciona ambos en un elemento resistivo bajo el modelo que usamos en clase. La analogía del agua ayuda a imaginar presión y caudal, pero el circuito se resuelve con magnitudes eléctricas. La corriente se expresa con frecuencia en miliamperios: 5 mA son 0,005 A. Convertir correctamente la unidad evita seleccionar una resistencia mil veces distinta.

Magnitudes que vamos a distinguir

Magnitud	Unidad	Qué pregunta responde
Voltaje	V	Diferencia entre dos nodos
Corriente	A o mA	Cuánta carga circula
Resistencia	ohm	Relación V/I del resistor
Potencia	W	Energía por segundo

Voltaje, corriente y resistencia con unidades

$$V_R = IR, \qquad P_R = V_R I = I^2 R$$

El LED tiene polaridad y una curva corriente–tensión. Para el cálculo inicial tomamos una caída de tensión aproximada del componente y dejamos el resto a la resistencia. Esa aproximación debe declararse: un LED rojo y uno azul no tienen necesariamente la misma caída. El valor que medimos en el GPIO también puede diferir del ideal cuando entrega corriente. El cálculo sirve para proponer el circuito y la medición sirve para contrastar la propuesta.

La ley de Ohm se aplica a la resistencia. El LED tiene una caída de tensión propia; no es una resistencia ideal.

Un cálculo que decide una pieza

Medimos tensión en paralelo con el componente. Para comenzar basta medir la caída en la resistencia y calcular su corriente. No se coloca el multímetro en modo corriente directamente entre alimentación y GND: esa conexión altera radicalmente el circuito. El docente muestra las puntas, la función y los puntos de medida antes de energizar. El registro debe indicar qué dos nodos se compararon, porque “medí 1,3 V” sin esa referencia no explica el circuito.

Ficha del primer LED

Dato	Predicción de aula	Dato del equipo
Resistencia	330 ohm	Leer y medir
Caída en resistor	1,3 V ideal	Medir nodos
Corriente	3,94 mA ideal	Calcular con V/R
Estado lógico	HIGH / LOW	Observar salida

LED rojo del ejercicio

$$R = \frac{V_{GPIO} - V_F}{I}, \qquad I_{330} = \frac{3.3 - 2.0}{330} \approx 3.94\,\mathrm{mA}$$

La resistencia recibe 1,3 V. R = 1,3 / 0,005 = 260 ohm. Con 330 ohm, la corriente ideal sería 1,3 / 330 = 3,94 mA. La potencia del resistor sería aproximadamente 5,1 mW. Son cifras del modelo de aula; se comprueban con las piezas reales.

Se elige un valor comercial razonable y se comprueba el conjunto, sin tratar el límite absoluto de un pin como corriente de trabajo.

El recorrido del programa hasta el pin

VS Code es el editor; PlatformIO organiza el proyecto y sus herramientas; Arduino es el framework que aporta funciones de programación. El archivo platformio.ini declara placa, plataforma y bibliotecas. El curso incluye una versión de referencia fijada para ESP32 clásico. Compilar convierte el programa en firmware; cargarlo lo transfiere a la placa; abrir el monitor serie permite observar mensajes. Un error de compilación y un circuito mal conectado requieren comprobaciones diferentes.

Fragmento del primer ejemplo

```
constexpr uint8_t LED = 23;
uint32_t last = 0;
bool on = false;
void setup() {
  pinMode(LED, OUTPUT);
}
void loop() {
  uint32_t now = millis();
  if (uint32_t(now-last) >= 500) {
    last = now;
    on = !on;
    digitalWrite(LED, on);
  }
}
```

Lee el fragmento. Predice una salida y cambia una sola condición antes de ejecutar.

Interpretación · El recorrido del programa hasta el pin

setup() prepara la ejecución y loop() vuelve a evaluar la lógica. El nombre LED ayuda a relacionar el esquema con un GPIO concreto. La función digitalWrite() ordena un nivel; el camino eléctrico determina qué ocurre con ese nivel. La versión completa c01_led también emite mensajes. En esta clase reconocemos el patrón temporal; en la tercera explicaremos por qué permite atender otros eventos. Antes de cambiarlo, predice cuánto dura cada estado y cuánto dura el ciclo completo.

Laboratorio: medir, ejecutar y explicar

Un parpadeo correcto demuestra que esa configuración produjo el comportamiento observado. Para aprender más, cambia una sola condición: tiempo, valor de resistencia o pin documentado. La misma comprobación se hace con el código y con el circuito delante. Si el LED permanece apagado, comienza por alimentación, referencia, polaridad y camino de corriente. Si los mensajes cambian pero la luz no, ya existe una pista que permite separar la ejecución del programa de su salida física.

Predicción y explicación

¿500 ms por cambio producen un ciclo de 500 ms?

Registrar una medida y explicar la relación V/R.

Práctica · Laboratorio: medir, ejecutar y explicar

1. Identifica módulo, placa y GPIO23. Revisa la configuración de referencia antes de compilar c01_led.

2. Dibuja el camino de corriente. Coloca LED y resistencia con la alimentación desconectada.

3. Comprueba continuidad del protoboard y polaridad. Pide una revisión cruzada del esquema.

4. Ejecuta primero la versión equivalente en Wokwi y localiza qué variable representa cada componente.

5. En hardware, mide tensión en el resistor y calcula corriente. Anota qué valor de resistencia utilizaste.

6. Cambia el tiempo de 500 a 250 ms. Predice el periodo total antes de observarlo.

7. Conserva esquema, código y una explicación individual del resultado.

Aplicación a drones y lectura de investigación

Un sistema UAV utiliza indicadores para distinguir alimentación, enlace, registro o estado del equipo. Un LED aislado puede indicar que hay energía, pero el software puede usarlo para comunicar otra condición. En una estación de banco, un patrón lento podría significar espera y otro representar una lectura pendiente. Esa convención debe estar documentada: el color no explica automáticamente el estado. El alumno propone un indicador para una función del mercado de MECRO y describe qué señal necesitaría para activarlo.

Del circuito sencillo al sistema

Hoy	Aplicación posterior	Qué añade el sistema
GPIO y LED	Indicador de estado	Condición documentada
GND y tensión	Interfaz de sensor	Niveles compatibles
Resistor calculado	Etapa de salida	Balance de potencia

Comprobación final

¿HIGH equivale a una fuente ideal que alimenta cualquier carga?

HIGH expresa un estado de salida dentro de límites eléctricos. La corriente y la carga deben ser compatibles con el GPIO y con el circuito.

Práctica guiada

1. Mostrar la placa y seguir el camino de corriente antes de abrir el programa.
2. Resolver el LED de 330 ohm y comparar con el resistor disponible.
3. Ejecutar c01_led y modificar su periodo.
4. Registrar una medida y explicar la relación V/R.

Aplicación en un banco para drones

Un indicador de estado en una estación de drones debe comunicar una condición mediante un circuito y un programa que se entiendan.

Explica una decisión con el esquema, el programa o un dato. ¿Qué queda sin comprobar?

Preguntas de cierre

1. ¿Dónde se aplica V = IR?
2. ¿500 ms por cambio producen un ciclo de 500 ms?
3. ¿Qué confirma compilar?

Respuestas y razonamiento

1. En el resistor del modelo; el LED se representa con su caída de tensión aproximada.

2. No. Encendido y apagado duran 500 ms cada uno: el ciclo completo dura 1 s.

3. Que el código se pudo construir para la configuración elegida; el circuito todavía se comprueba aparte.

Entrega de la clase

Esquema del LED, cálculo con unidades, primer firmware y registro de observación.

Identifica versión, unidad y procedencia. Conserva la evidencia y explica qué probarías después.

Programación y entradas digitales: botones que significan eventos

Organizar un programa con funciones y detectar una pulsación sin confundir nivel, transición y rebote.

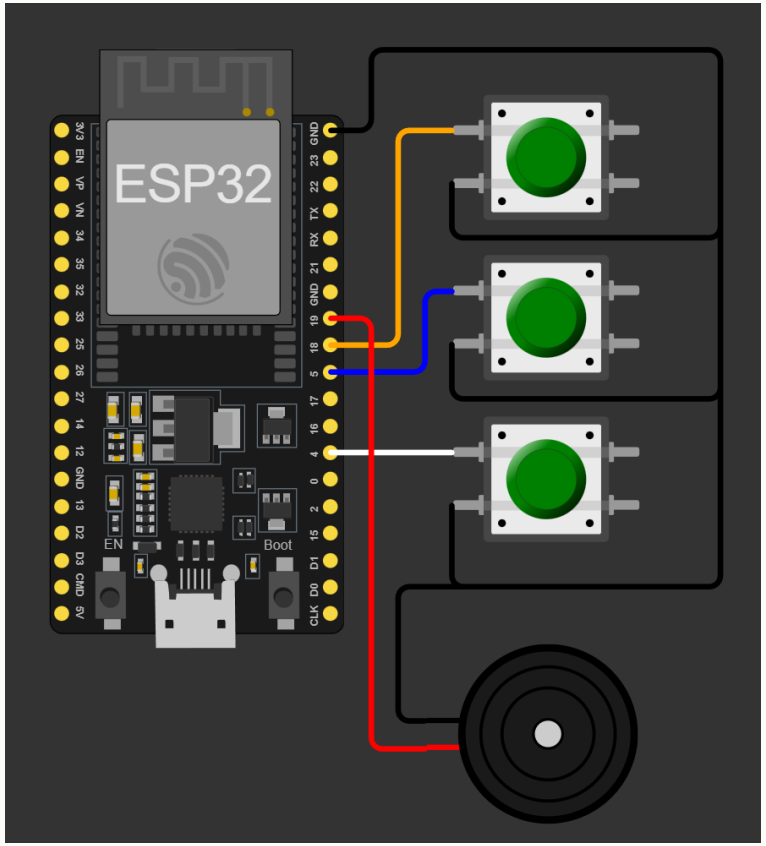
Los mandos y los avisos de una estación de drones necesitan interpretar acciones humanas con estados definidos.

EN LÍNEA 2 HORAS ESP32 CLÁSICO

Entrada, decisión y salida

Un programa de hardware transforma observaciones en decisiones. El pulsador es la entrada; una condición representa la decisión; una luz o un sonido comunica la salida. Antes de escribir instrucciones, define el comportamiento: “mientras mantengo pulsado, la luz permanece encendida” es diferente de “cada pulsación cambia el estado”. La primera regla usa un nivel; la segunda necesita reconocer un evento. Si no se aclara esa diferencia, el programa puede parecer defectuoso aunque ejecute exactamente lo que se escribió.

Entrada, decisión y salida



¿El programa necesita el nivel del botón o una pulsación? Señala entrada y salida.

MECRO, proyecto 02. Esquema original del laboratorio para reconocer la arquitectura en Wokwi. Para montaje físico se aplica la tabla y las correcciones de esta lectura.

Interpretación · Entrada, decisión y salida

El proyecto del piano permite observar esta cadena con tres entradas. Cada botón selecciona una acción. El dado digital añade una variable que conserva un resultado después de la pulsación. Reutilizaremos ambos como ejercicios de programación; no es necesario repetir dos montajes completos para enseñar el mismo concepto. Cada estudiante modifica una regla y debe explicar cuándo se evalúa.

Tipos, condiciones y funciones que ayudan a pensar

Una constante nombra una elección que no cambia durante la ejecución: por ejemplo, el número de pin. Una variable conserva información que sí puede cambiar. bool representa dos estados lógicos; un entero representa un conteo; float permite valores fraccionarios con precisión limitada. El tipo también afecta las operaciones. Si dividimos dos enteros, el resultado sigue reglas de aritmética entera. Escribir la unidad junto al nombre, como tiempo_ms o distancia_cm, reduce confusiones sin exigir un lenguaje más complejo.

Una intención y su representación

Intención	Ejemplo	Uso
Pin fijo	constexpr uint8_t BTN=18	Relacionar esquema y código
Estado	bool encendido=false	Conservar una decisión
Tiempo	uint32_t ahora=millis()	Medir intervalos
Magnitud fraccional	float distancia_cm	Expresar una lectura

División que cambia el resultado

¿Cuánto vale 5/2 en una operación entre enteros y cómo obtienes 2,5?

Entre enteros da 2. Una expresión como 5.0f/2.0f conserva la fracción. La decisión depende de qué magnitud quieres representar.

Pull-up y rebote: dos problemas diferentes

Una entrada sin referencia puede responder a interferencias y mostrar estados variables. INPUT_PULLUP habilita una resistencia interna que tiende a llevar el pin a HIGH. El pulsador lo conecta a GND al presionarlo, así que pulsado se interpreta como LOW. Esa inversión es una convención del circuito. No se corrige cambiando cables al azar: se relacionan nivel físico y significado en una tabla de verdad.

Pulsador entre GPIO18 y GND

Situación	Lectura	Significado elegido
Abierto	HIGH	Reposo
Presionado	LOW	Solicitud
Lecturas alternantes breves	HIGH / LOW	Posible rebote

Una pulsación, varios cambios

Un contador aumenta cinco veces al tocar una vez el botón. ¿Basta cambiar INPUT por INPUT_PULLUP?

Primero se necesita una referencia válida. Si ya existe, hay que distinguir transición y rebote: contar una transición estable, no cada recorrido de loop() con el botón pulsado.

Lectura del antirrebote incluido

raw es la lectura instantánea. rawPrevious permite reconocer que cambió. changed conserva cuándo se produjo el cambio más reciente. stable representa lo que el programa acepta después de la espera. Durante esa espera, loop() puede seguir atendiendo otras tareas. Al separar estas variables evitamos llamar “estado del botón” a tres cosas que no significan lo mismo. La versión completa emite PRESS y RELEASE para observar ambas transiciones.

Núcleo de c02_boton

```
bool raw = digitalRead(BTN);
if (raw != rawPrevious) {
  rawPrevious = raw;
  changed = now;
}
if (uint32_t(now-changed) >= 25
    && raw != stable) {
  stable = raw;
  digitalWrite(LED, stable == LOW);
}
```

Lee el fragmento. Predice una salida y cambia una sola condición antes de ejecutar.

Interpretación · Lectura del antirrebote incluido

El piano original usa prioridades con if y else if: si se pulsan dos botones, la primera condición verdadera domina. Esa decisión puede ser correcta para un instrumento monofónico. Para una interfaz de mando hay que documentar qué ocurre con órdenes simultáneas. El alumno prueba dos entradas y describe el resultado antes de proponer otra prioridad. Programar también consiste en resolver esas situaciones límite.

Laboratorio y reto del dado

La simulación del dado sirve para comprender condiciones, variables y presentación. Un display real puede ser de ánodo o cátodo común y necesita limitar corriente por segmento según su montaje. La ilustración original del laboratorio no sustituye esa revisión. Por eso el reto obligatorio evalúa primero la lógica y la tabla de segmentos. La construcción física se realiza con un esquema eléctrico completo revisado por el docente.

Predicción y explicación

¿El pull-up elimina el rebote?

Resolver la prioridad del piano y el reto lógico del dado.

Práctica · Laboratorio y reto del dado

1. Conecta GPIO18 a GND mediante un pulsador; utiliza INPUT_PULLUP.

2. Ejecuta c02_boton y relaciona PRESS y RELEASE con la acción física.

3. Mantén pulsado dos segundos: verifica que no aparezcan nuevas pulsaciones aceptadas.

4. Añade un contador que aumente únicamente al aceptar stable == LOW.

5. En el piano del laboratorio, predice la prioridad de dos teclas y compruébala.

6. Como reto, usa el evento para solicitar un nuevo valor del dado y conserva el resultado.

7. Documenta las resistencias por segmento si se propone montar un display desnudo en hardware.

Un contrato para las entradas humanas

En un banco relacionado con UAV, una pulsación puede solicitar una prueba, reconocer un aviso o cambiar la pantalla. Define el estado inicial, la acción permitida y qué sucede si el botón queda retenido. La interfaz debe poder explicarse a otra persona. Una buena prueba incluye pulsación breve, larga, doble, simultánea y al iniciar el equipo. Ninguna de esas pruebas requiere un dron en vuelo; se aprende con una entrada y una salida observables.

Casos de prueba de la interfaz

Prueba	Resultado que se define
Mantener pulsado	Un evento o acción sostenida
Dos botones	Prioridad o combinación
Pulsado al arrancar	Estado inicial documentado
Rebote	Una transición aceptada

Interpretación · Un contrato para las entradas humanas

Investiga la diferencia entre un botón con resistencia externa y un pin con resistencia interna habilitada. En el ESP32 clásico, GPIO34–39 son entradas y carecen de pull-up o pull-down internos. La elección del pin, por tanto, cambia el circuito necesario. Relaciona esa información con el hecho de que esos mismos pines pueden resultar útiles para lecturas analógicas. El nombre “GPIO” no promete exactamente las mismas funciones en todos los números.

Práctica guiada

1. Mostrar una pulsación larga y pedir la diferencia entre nivel y evento.
2. Rastrear raw, changed y stable con una tabla temporal.
3. Comprobar c02_boton y añadir el contador de eventos.
4. Resolver la prioridad del piano y el reto lógico del dado.

Aplicación en un banco para drones

Los mandos y los avisos de una estación de drones necesitan interpretar acciones humanas con estados definidos.

Explica una decisión con el esquema, el programa o un dato. ¿Qué queda sin comprobar?

Preguntas de cierre

1. ¿LOW siempre significa apagado?
2. ¿El pull-up elimina el rebote?
3. ¿Por qué no contar cada loop() en LOW?

Respuestas y razonamiento

1. Su significado depende del circuito; con pull-up puede representar un botón pulsado.

2. Fija el nivel en reposo; el rebote se trata con otra estrategia.

3. Porque una pulsación sostenida atraviesa muchas iteraciones.

Entrega de la clase

Código del botón, tabla nivel/evento y cuatro pruebas de interfaz.

Identifica versión, unidad y procedencia. Conserva la evidencia y explica qué probarías después.